

Návrh testovací strategie

1. Popis aplikace

Testovaným systémem je aplikace **Meeting Room Reservation System**, která slouží k rezervaci zasedacích místností v kancelářském prostředí. Aplikace umožňuje uživateli prostřednictvím REST API vytvořit novou rezervaci, potvrdit rezervaci a zrušit existující rezervaci.

Při vytváření rezervace systém ověřuje několik business pravidel. Kontroluje, zda zadaná místnost existuje, zda počet osob nepřekračuje kapacitu místnosti, zda je časový interval rezervace platný, zda rezervace probíhá v pracovních dnech a v pracovní době, zda délka rezervace splňuje minimální i maximální požadavek, zda čas odpovídá 15minutovým intervalům a zda nová rezervace nekoliduje s již existující rezervací.

Na základě počtu osob systém rozhoduje o stavu rezervace. Menší rezervace (do 4 osob) jsou automaticky potvrzeny (CONFIRMED), zatímco větší rezervace přecházejí do stavu PENDING a vyžadují dodatečné potvrzení.

Při potvrzení rezervace systém ověřuje existenci rezervace a její aktuální stav. Při rušení rezervace systém kontroluje existenci rezervace a zajišťuje, že rezervace není již zrušená.

Aplikace je implementována jako REST aplikace ve frameworku Spring Boot. Je rozdělena do vrstev controller, service, repository a model. Největší riziko systému spočívá v nesprávné validaci časových omezení, chybné detekci konfliktů mezi rezervacemi a nesprávném řízení stavů rezervace.

2.1 Přehled částí aplikace

Aplikace je rozdělena do následujících částí:

Controller vrstva

Zajišťuje komunikaci s klientem pomocí HTTP požadavků. Obsahuje endpointy pro vytvoření, potvrzení a zrušení rezervace. Odpovídá za přijetí vstupních dat a předání požadavků do service vrstvy.

Service vrstva

Obsahuje hlavní business logiku aplikace. Zejména zajišťuje:

- validaci vstupních dat rezervace,
- kontrolu existence místnosti,
- kontrolu kapacity místnosti,
- kontrolu časových pravidel rezervace,
- kontrolu konfliktu s existujícími rezervacemi,
- změnu stavu rezervace při jejím potvrzení nebo zrušení.

Repository vrstva

Zajišťuje ukládání a načítání dat. V aplikaci je použita in-memory implementace repository. Součástí této vrstvy je také logika pro zjištění časového konfliktu mezi rezervacemi.

Model vrstva

Obsahuje datové objekty a enum typy používané v aplikaci:

- Reservation
 - Room
 - ReservationStatus
-

2.2 Prioritizace částí aplikace

Části aplikace byly prioritizovány podle rizika a dopadu případné chyby na chování systému.

Vysoká priorita – Service vrstva (ReservationService)

Tato vrstva obsahuje hlavní business logiku aplikace včetně validačních pravidel a přechodů stavů rezervace. Chyby v této vrstvě mohou způsobit přijetí neplatné rezervace, ignorování kapacitních omezení nebo povolení rezervace mimo pracovní pravidla.

Vysoká priorita – logika konfliktů rezervací v repository vrstvě

Detekce konfliktu rezervací je kritická část systému. Chyba v této logice může vést k duplicitní rezervaci stejné místnosti ve stejném čase.

Střední priorita – Controller vrstva

Controller vrstva neobsahuje složitou business logiku, ale je důležitá pro správné zpracování HTTP požadavků a odpovědí.

Nízká priorita – Model vrstva

Modely a enum typy neobsahují složitou logiku. Jedná se převážně o datové struktury.

2.3 Test levels

Aplikace bude testována na následujících úrovních:

Unit testy

Unit testy budou zaměřeny na izolované testování business logiky ve třídě `ReservationService`. Budou ověřovat jednotlivé validační podmínky, mezní stavy a větve logiky.

Například:

- neplatné časové intervaly,
- čas rezervace mimo 15minutové intervaly,
- překročení kapacity místnosti,
- rezervace o víkendu,
- rezervace mimo pracovní dobu,
- příliš krátkou rezervaci,
- příliš dlouhou rezervaci,
- konflikt rezervací,
- automaticky potvrzenou rezervaci,
- rezervaci ve stavu PENDING,
- úspěšné potvrzení rezervace,
- pokus o potvrzení rezervace v neplatném stavu,
- rušení již zrušené rezervace,
- chování při neexistující rezervaci nebo místnosti.

Pro izolaci závislostí budou použity mock objekty `RoomRepository` a `ReservationRepository` pomocí knihovny Mockito.

Integrační testy

Integrační testy budou ověřovat spolupráci mezi jednotlivými vrstvami aplikace, zejména mezi controller, service a repository vrstvou. Zaměří se na správné fungování REST endpointů a na ověření, že systém vrací očekávané odpovědi při validních i nevalidních požadavcích.

Procesní testy

Procesní testy budou pokrývat hlavní uživatelské průchody aplikací. Budou vytvořeny alespoň pro tyto procesy:

- vytvoření rezervace,
- potvrzení rezervace,
- zrušení rezervace.

U každého procesu budou zohledněny jak úspěšné průchody, tak chybové větve, například konflikt rezervace, neexistující místnost, neplatná data, pokus o potvrzení rezervace v neplatném stavu nebo pokus o opětovné zrušení již zrušené rezervace.

Testy budou odvozeny z procesních diagramů a zpracovány s úrovní pokrytí **TDL 2**.